

C Programming Language

- * C යනු; Dennis Ritchie විසින් Bell Laboratories හි 1972 දී
ව්‍යවහාරික කරන ලද general-purpose programming language වෙයි.
- * C ආවේණික කිරීමේ කළමනාකරණය කරන ප්‍රධාන ලේඛන දෙකක් වන්නේ.
 - i) codes ලිවීමේ කළමනාකරණ text editor වෙයි (Notepad, Notepad ++, Sublime text ...) හෝ IDE (Integrated Development Environment) වෙයි (E.x. Code:Blocks, visual studio code, Eclipse ...) වන්නේ.
 - ii) C තුළින් ලියූ code වෙත machine language වෙත
translate කිරීමේ GCC (GNU C Compiler) නම්
compiler වෙයි වන්නේ.
- * C code ආකර්ෂණය source code file වෙයි save කිරීමේ දී, file extension
වෙයි ලෙස ".c" යොදයි.

E.x. myfirstprogramme.c
file name extension

Syntax of C

- * වෙනම ව්‍යාකරණය වූනි ව්‍යවහාරික (grammar) ආකර්ෂණය, programming language
වෙයි වූනි; එම ව්‍යාකරණයේ ආවේණිකයන් සිහිපත් වීනි Syntax ලෙස හඳුන්වයි.
- * පහත දැක්වෙන්නේ C තුළින් ලියූ පළමු code වෙයි.

```
#include <stdio.h>

int main () {
    printf("Hello World!");
    return 0;
}
```

- ⊙ මෙහි පළමු පේළියේ වූනි #include <stdio.h> මෙහි දැක්වෙන්නේ
"header file library" වෙයි. මෙය input හා output functions කළමනාකරණය
කරයි.
 - ** මෙය පළමු විට C programme වෙයි දී දැක්වීමට ලැබෙන
නොවෙයි.

- ⊙ main() යනු; function වෙයි. function වෙත වූයේම run වෙන්නේ
(execute = run) නමුත් වර්තමාන (curly braces/curly brackets) තුළ
මෙම code වෙයි.

* int යනු; data type වෙති. (integer යන්න දැක්වේ. ඉරිදි/නිල සංකේත)

* printf() යනු; predefined function වෙති. මෙය තත්ත්වය තුළ සොයා ගත් output එම ස්ථානවලට දැක්වේ.

E.x. ඉහත දැක්වීමට දී;

Hello World! ලෙස output කරයි.

* return 0 යන්නෙන් main() function වෙතින් නිකුත් කරන ද, එවැනි return value = 0 යන්න දැක්වේ.

විද්‍යා: C compiler එකින් code වෙතින් compile කිරීමේ දී නිකුත් (whitespaces) සහතික කරයි.

මේ හේතුවෙන්; තමා හොඳින් කියවා ගැනිය හැකි (more readable) code වෙතින් ස්වල්ප whitespaces සමඟ ලබා ගන්න.

❖❖ code line වෙතින් නිකුත් කරන semicolon(;) නිකුත් කිරීමට සිදුවිය යුතුය. එවැනි, ඉහත code line වෙතින් නිකුත් කළ කළ දැක්වීම ලබයි.

Outputs

* values හෝ text වෙතින් output කිරීමට, printf() function වෙතින් යොදා ගන්න.

* text වෙතින් output කිරීමට තමා; එය සඳහා දෙගුණ (double quotes) තුළ යොදා ගන්න.

E.x. printf("Hello, How are you?");

* එකතු printf() functions කිහිපයක් යොදා ගැනිය හැකි වුවත්; මෙම ක්‍රමයේ දී output වෙතින් ඉහත වෙතින් නිකුත් කරන සෑම දෙයක්ම එකම රේඛාවක, එකතු වෙතින් ලබයි.
(all outputs appear in the same line)

E.x. printf("Hello");

printf("What is your name?");

printf("How old are you?");

⇓ Output

Hello What is your name? How old are you?

* new line වෙතින් මෙම outputs ලබා ගැනීමට තමා; \n escape character වෙතින් යොදා ගන්න.

E.x. printf("Hello");

printf("What is your name? \n");

printf("How old are you?");

Output
=>

Hello What is your name?

How old are you?

Note : C language ໃຊ້ ຄຳ ຂໍ້ ຂັດຂວາງ ຄຳ ຂັດຂວາງ ໃນ ຂໍ້ ຂັດຂວາງ ໃນ ຂໍ້ ຂັດຂວາງ.

Escape character	Result	Description
\'	'	insert a single quote in a string
\''	''	insert a double quote in a string
\\	\	insert backlash in a string
\n	new line	insert a new line
\t	Tab	insert a tab space
\0	null	insert null value

E.g.

- 1) `printf("Hello \ ' Sri Lanka \ ' ");` $\Rightarrow$ Hello 'Sri Lanka' single quotes
- 2) `printf("Hello \" Sri Lanka \" ");` $\Rightarrow$ Hello "Sri Lanka" double quotes
- 3) `printf("Apple \ Orange");` $\Rightarrow$ Apple \ Orange backlash
- 4) `printf("Apple \n Orange");` $\Rightarrow$ Apple
Orange } new line
- 5) `printf("Apple \t Orange");` $\Rightarrow$ Apple Orange
tab space

Comments

- ② C code ଥିବାକୁ $\frac{1}{2}$ ବଢ଼ାଇବା code ଲେଖକଙ୍କ ପଢ଼ିବା ପାଇଁ ସହଜ କରିବା
 ଯେଉଁ more readable କରିବା comments ଦେଇ ଥାଏ।
- * comments ଥିବାବେଳେ, code ଥିବା ଲିନିଆ/ସଂକ୍ଷିପ୍ତ ମୂଳ ଲେଖା edit କରିବା
 ସୁବିଧା ଯେ. whitespaces ଥିବା comments, compiler ଦ୍ଵାରା ଲେଖକଙ୍କୁ (ignore)
 କରେ। ଏହା କିଛି; **comments execute ହୋଇଥାନ୍ତି।**

★ ★ Outputs ଏବଂ comments ସେ ମୁଁ ୧୨ ~~କୋଡ୍~~ କୋଡ୍‌ଗୁଡ଼ିକ

* Comments ଘରର ଉପର 2 ଫି.

- i.) Single line comments.
- ii.) multiline comments.

④ Single line comment એવાની યજ્ઞના ચર્ચિત્તે ભવિષ્યે ડાલના.

// This is a single line comment

- ④ Single line comment એટલે **double slash** પછીનું પ્રવર્તેલ છે. એક નવું લેઈન (in a new line) લખવામાં આવે; comment એટલે પ્રવર્તેલ એવું, એ લેઈન પ્રવર્તેલ છે.

E.x. // This is comment ← comment line (not be executed)
printf("Hello"); ← executable code line

- ⑧ ଅଧିକ ଗର୍ଭ ଥିବାର ଚେତନା comment କେହିକି ଆଣିଛନ୍ତି ସୁ.

E.x. `printf("Hello");` // This is comment

executable code part comment part

- ④ Multiline comment शेखर उपना उर्फ वेदिका यश.

```
/* This  
is a  
multiline  
comment */
```

- (*) / * ઝાલ * / જગત ભરના ભણે સર્વિસમાં દૂરા comments ની કુલાક
ચર્ચા compiler થી ignore કરી શકાય.

Variables

- * variables (ବିଭେଦ) କି; କିଛି value ଥିବା (like numbers or characters) store କରିବା ପାଇଁ computer memory ଥିବା ଅବସ୍ଥାକୁ କୁହାଯାଏ।

- * data type એક group છે, જે variables સંબંધિત સમૂહ રજીસ્ટર કરે.

Ex. i) int $\Rightarrow$ integers (whole), without decimals (123, 202, 0, -5)

iii) float $\Rightarrow$ floating point numbers, with decimals (19.99, 0.00, -5.7)

iii) char $\Rightarrow$ single characters. These characters are surrounded by single quotes ('a', 'c')

- * variable ଯେଉଁ ଗଠିତ ହୁଏ, variable ଯେଉଁ create କରି ଏହା
variable ଯେଉଁ **declare** କରି ତେଜ ଅନ୍ତର୍ଭୁକ୍ତ.

- * variable declaration হল এই তথ্য দেয় কতটা বড় হবে।

data type variable name = value ;

* ඉහත උදාහරණ කෙරෙහි.

E.x. 1) `int myNum = 15;`
2) `char myChar = 'a';`
3) `float time = 20.12;`

① variable එකක් create කිරීමේ දී ඉහත වගන්තිවලට ප්‍රධාන සාරාංශය ලෙසින් සිදු වේ.

(i.) **Declaration** ⇒ data type එකක් කෙසේ, variable name එකක් කො
දී variable එක නිර්මාණය කිරීම.

(ii.) **Assigning** ⇒ නිර්මාණය කර variable එකට value එකක් ලබා දීම.

* ඉහත උදාහරණ වල එරිදි තනි පේළියක දී මෙය සිදු කර ඇති අතර, ඉවහරය තව
පේළි දෙකකින් වුව ද, සිදු කිරීමේ නිසිදායකයක් නැත.

E.x. `// Declare a variable`

`int myNum;`

`// Assign a value to the variable`

`myNum = 10;`

Format Specifiers

* Format specifiers, `printf()` function එක කමක කෙරෙයි.

* variable එකක store කර ඇත්තේ කුමන type එකට ඉහළ data ද, කවි
මෙහිදී compiler ට දැනුම් දේ.

* format specifier එකක් ඉරිමනු වන්නේ **% (percentage sign)** වල අතර,
මුත් ඉන් අනුමාන කෙරෙයි.

** `printf()` මුළු value එකක් කෙසේ output කිරීමට මෙය ඉවහරය හොඳයි!
මුත්, variable එකක් නිර්මාණය කර, එය output කිරීමට මෙය ඉවහරයක්
වේ.

E.x. output the value of an **int** variable, use the format specified
%d surrounded by double quotes, inside the `printf()`
function.

1) `int myNum = 15;`

`printf("%d", myNum); // Outputs 15`

* To print other types ⇒ use **%c** for char and **%f** for float.

Try this one

```
int myNum = 15;  
float myFloat = 5.99;  
char myChar = 'c';
```

```
printf("%d\n", myNum);  
printf("%f\n", myFloat);  
printf("%c\n", myChar);
```

★ text એવાજ કદા variables combine કરીને હું હ, ભેગા બેઠકેક ઢાલો.

E.x.

```
int myNum = 15;  
printf("my favourite number is: %d", myNum);
```

★ એવાજ printf() function એકા જુદા એવાજ data types એ variables કદાજ કરી બેઠકેક ઢાલો.

E.x.

```
int myNum = 15;  
char myChar = 'c';  
printf("My number is %d and my character is %c", myNum, myChar);
```

⇓ output

My number is 15 and my character is c

★ variables એવાજ values print કરીને હ, ભેગા જુદા બેઠકેક ઢાલો.

E.x.

```
printf("My favourite number is: %d", 15);
```

⇓ output

My favourite number is : 15

Note : ભેગા જુદા એવાજ જુદાજ એવાજ.

% variables એવાજ re-usability એકા કરી બે.

★ variable એવાજ એકા value એકા એવાજ કરી કદાજ, એક variable name એકા એકા એકા value એકા એવાજ એકા.

E.x.

```
int firstNum = 5; // firstNum is 5
```

```
firstNum = 10; // Now it is 10
```

```
printf("%d", firstNum); // Outputs 10
```

★ એક variable એકા value એકા એકા variable એકા assign કરી કદાજ કરી કદાજ ઢાલો.


```
E.x. int myNum = 15;  
      int anotherNum = 20;
```

myNum = anotherNum; // Assigning the value of anotherNum to myNum

```
printf("myNum"); // Outputs 20  
printf("%d", myNum); // Outputs 20
```

* Empty variable ସୂଚକ value ସୂଚକ assign କରିବେ ϵ , ତେଣୁ ସୂଚକ ନା ହୁଏ.

```
E.x.  int firstNum = 10;
      int secondNum;
```

```
secondNum = firstNum; // Assigning the value of firstNum to  
                        secondNum
```

```
printf("%d", secondNum); // Outputs 10
```

* **data type** වෙනස් **variables** තිබියදී **එකම** **value** **බෙදා** **බෙදා**

★★ ଭେକ୍ଟର କ୍ଷେତ୍ର () ଚାରିଟି ଭେକ୍ଟର ଭେକ୍ଟର (comma separated list) ରେ 25 variables ଥାଏ.

```
Ex. int x = 5, y = 6, z = 10;
    printf("%d", x + y + z); // Outputs 21
```

* **କଣ୍ଠର declare କର**, **ପରେ ଭାରିକେସ' values assign କରିବି** ଆକାଉଣ୍ଟରେ
କରାଯିବ.

```
Ex.    int x, y, z;  
        x = y = z = 50;  
        printf("%d", x + y + z);
```

↓ output
150

```
int x, y, z;  
x = 5;  
y = 6;  
z = 10;  
printf("%d", x + y + z);
```

11 outputs

* C programming language is case-sensitive is.

එනම්; 9෮ කින් (variable එකක්) ඇතිවීමට කොපමණ $unique$ විය යුතුයි.

* ଥିବା ଅନ୍ୟ variables ଯେଉଁ ସେ ବିଶ୍ଳେଷଣର ନିର୍ଦ୍ଦେଶ ଦେଇ ପାରିବେ; ଥିବା ଅନ୍ୟ ହୁଏ, capital-simple letters ଯେଉଁ ଶୂନ୍ୟ ଅଟେ
ଯେଉଁ variables ରେ ଆଧାର ମିଳେ.

E.g. myNum and MyNum, compiler ଦିଅନ୍ତି ଧାରାବାଚକର ନାମର variables ଗୁଡ଼ିକର ଗୁଡ଼ିକ.

* variables should have unique names, **identifiers** are used to identify.

* identifiers are used to identify variables and functions. They are used to identify variables and functions.

* It is **recommended to use descriptive names** in order to create understandable and maintainable code.

E.x. first number should be;

'firstNumber' instead of 'firstNum' or any other.

* variables should be **declared** before they are used, and they should be declared before they are used.

⊛ variable name should be english letters (capital or small), numbers (whole numbers) and underscore (_) are allowed.

⊛ variable name should be **lowercase** and **underscore** should be used.

⊛ variable names are **case-sensitive**. (myNum and mynum are different)

⊛ variable name should not contain **whitespaces** and **special characters** (!, ?, #) etc.

⊛ variable names should not be **C language** or **reserved words / key words** (int, float, ...) etc.

Data types

* C has **different variable data types** and they are used to store data. The output is given by printf() function and format specifiers are used.

* There are **4 data types** in C.

int ⇒ 2 or 4 bytes in size. It is used to store integer values.

float ⇒ 4 bytes in size. It is used to store floating point values. It is used to store 6-7 digits after the decimal point.

double ⇒ 8 bytes in size. It is used to store double precision floating point values. It is used to store 15 digits after the decimal point.

char ⇒ 1 byte in size. Single character / letter / number / ASCII value is stored.

* මෙම data types සඳහා යොදාගන්නා basic format specifiers වන්නේ පහත වේ.

%d හෝ %i $\Rightarrow$ for int type

%f හෝ %F $\Rightarrow$ for float type

%lf $\Rightarrow$ for double type

%c $\Rightarrow$ for char type

%s $\Rightarrow$ used for strings.

char type

* Single character වලින් store කිරීමට 'char' type වෙත යොමු වේ.
එක් යොදාගන්නා character වෙත single quotes තුළ සිටුවිය යුතුය.

E.x. char myChar = 'S';
printf("%c", myChar);

* char type වෙත අවම වශයෙන් ASCII values 2, සිටුවිය යුතුය. නමුත්, මෙය single quotes යොදාගෙන පෙන්විය යුතුය.

Ex: char a = 65;
printf("%c", a); // Outputs A

* single quotes තුළ ඇති character වලින් පමණක් වෙනස්කිරීමට යොදාගෙන, එම වෙනස්කිරීමේ output ලබා දිය යුතුය.

E.x. char myChar = 'Lanka';
printf("%c", myChar); // Outputs a

Note: වෙනස්කිරීමට char data type යොදා ගැනීමෙන් Errors ලැබිය යුතුය. එය strings යොදාගෙන යුතුය.

number types

* numbers store කිරීමට යොදාගන්නා ප්‍රධාන data types තුනකි.

(i.) int $\Rightarrow$ පූර්ණ සංඛ්‍යා ගණනය කළ හැකිය.

(ii.) float $\Rightarrow$ දශමයෙන් 6-7 දක්වා වන දශම සංඛ්‍යා ගණනය කළ හැකිය.

(iii.) double $\Rightarrow$ දශමයෙන් 15 දක්වා වන දශම සංඛ්‍යා ගණනය කළ හැකිය.

⊗ පැහැදිලි කිරීමට වැදගත් වන්නේ නිවැරදිව ගණනය කිරීමට float වලට double වැඩිපුර ස්ථාන ඇත නමුත් float (4 bytes) වලට double (8 bytes) memory වැඩිපුර ලැබේ.

* විද්‍යාත්මක සංඛ්‍යා (scientific numbers) නිරූපණයට ද, float හෝ double data types යොදාගත හැක. මෙහි දී දශයේ බලය දැක්වීමට **e** හෝ **E** යොදා ගනී.

E.x. float f1 = 35e3;

double d1 = 12E4;

printf("%f\n", f1);

printf("%lf", d1);

return 0;

Output

35000.000000

⇒

120000.000000

* ඉහත ද්‍රව්‍යයන් සලකුම් වි; floating point number වෙත output කරන විට; දශමයට 6 කට සංඛ්‍යාව දැක්වීම බව නිරූපණය වේ.

* ඉහත ඉවසන දශමයට 6 ආසන්නයක් වෙත output කර ගැනීමට **dot(.)** සමඟ ඉවසන දශමයට 6 ආසන්නයක් යොදා ගනී. මෙය **set the decimal precision** වේ.

E.x. float myFloat = 3.5;

printf("%.f\n", myFloat); // Outputs 3.500000

printf("%.1f\n", myFloat); // Outputs 3.5

printf("%.3f\n", myFloat); // Outputs 3.500

Get Memory Size

* ඔබ කළහොත් ඉංජිනේරුයේ සහ data types විෂයේ computer memory වෙනස් වන දැක්වීම ඉටු ප්‍රමාණයක් ඉල්ලා කර ගනී.

int ⇒ 2 or 4 bytes

float ⇒ 4 bytes

double ⇒ 8 bytes

char ⇒ 1 byte

* යම් data type වෙනස් හෝ variable වෙනස් size වෙන bytes වලින් ලබා ගැනීමට **sizeof operator** වෙත යොදා ගනී.

E.x. int myInt;

float myFloat;

char myChar;

Output

4

printf("%lu\n", sizeof(myInt));

⇒

4

printf("%lu\n", sizeof(myFloat));

1

printf("%lu\n", sizeof(myChar));

Note: %d, %f, %c ඔබ්බට %lu යන format specifier එක ඔබ්බේ
සොයාගෙන ඇත. ඒ ඔබ්බේ තර්ජන, sizeof operator එක ඔබ්බේ
“long unsigned int (%lu)” type එකට ඇද value එකක් return
කරනු ඇතැයි compiler එකින් ප්‍රකාශය කිරීමයි.

(2) data types ରେ size ଥିବା ହିସାବରୁ ସଫ୍ଟୱେର ଡେଭଲପର, memory usage ଥିବା ଏବଂ performance ଥିବା ଚିନ୍ତା କରି ନିୟମିତ ଭାବେ ଚିନ୍ତା କରନ୍ତି।
 ଉଦାହରଣ ସ୍ୱରୂପ ଡାଟା ଟାଇପ ଥିବା କ୍ଷେତ୍ରରୁ ଡାଟା ଟାଇପ ଚିନିଷି ~~କରି~~
 memory save କରିବା ଉଚିତ, high performance programme ଡେଭଲପର
 ଡାଟା ଟାଇପ ଡାଟା ଟାଇପ ଓ ଡାଟା ଟାଇପ।

Type Conversion

* ମୂଳାତମ ମୂଲ୍ୟରୂପେ ଏହା data type ରୂପେ value ରୂପେ କିମ୍ବା data type ରୂପେ convert କରିବା ମଧ୍ୟ ସମ୍ଭବ। ଏହାକୁ type conversion କୁହାଯାଏ।

★ Type conversion ଦୁଇ ଯୋଗ୍ୟ ଶେ!

(i) Implicit Conversion (automatically)

(ii) Explicit Conversion (manually)

Implicit Conversion

* Compiler converts automatically int into float. float is int with decimal part.
 Example: 100 is int, 100.0 is float. 100 is converted to 100.0.
 Example: 100.0 is float, 100 is int. 100.0 is converted to 100.

E.x. `float myFloat = 9;` ← int value float type variable
`printf("%f", myFloat);` // Outputs 9.000000

⑤ આજી જીવનમાં જે કંઈક કરવું હોય તેની સાથે જોડાયેલી બધી જાણકારી એક જગ્યાએ રાખવી.

```
int myInt = 9.99;
```

```
printf("%d", myInt); // Outputs 9
```

* ହେଉ ଯୁଗେ ଯୁଗେ
ପୁରୁଷର ଶ୍ରଦ୍ଧା ଓ ମନା

Explicit Conversion

★ value ରେ ପ୍ରତିରୂପଣ parentheses () ଏବଂ data type ରେ ଉଲ୍ଲେଖ manually କରାଯିବା ଲାଗି.

Ex. 1.) float sum = (float) 5/2;

```
printf("%f", sum); // Outputs 2.500000
```

2.) `int num1 = 5;`

```
int num2 = 2;
```

```
float sum = (float) num1 / num2;
```

```
printf ("%f", sum);
```

Output
⇒

2.500000

Constants

- * variable value එකක් වෙනස් කළ හැකියාවක් ලෙස නිර්ණය කරමින් **const keyword** එක කොටස ලෙසි. එනම්; එම variable එක **constant** එකක් ලෙස හැඳින්වේ.

E.x. `const int myInt = 10;` // myInt will always 10
`myInt = 5;` // this may cause an Error!

- * කාලාන්තරයේ නිශ්චිතව වෙනස් නොවන දේ දැක්වීමට constants කොටස ලෙසි.

E.x. `const int minutesPerHour = 60;`

- * constant එකක් නිර්ණය කරන ඉවැස්ටාමේන්ට් එක value එකක් assign කිරීම ඉවැස්ටාමේන්ට් වේ.

E.x. `const int myInt;`
`myInt = 10;` // this may cause an Error!

- ⊕ more readable code එකක් ලබා ගැනීමට අප C programmers සිතුවන **good practice** එකක් ලෙස constants දැක්වීමේ දී **uppercase / capital** ඉවැස්ටාමේන්ට් දැක්විය ලෙසි.

E.x. `const int MYINT = 10;`

Operators

- * විවිධ ගණිත කර්ම සෑම නිර්ණය වීමට යොදා ගනිති.
- * ඉවැස්ටාමේන්ට් 5 ක් ඇත.

- 1.) Arithmetic operators
- 2.) Assignment operators
- 3.) Comparison operators
- 4.) Logical operators
- 5.) Bitwise operators

Arithmetic operators

Operator	Name	Description	Example
+	Addition	Adds together two values	$x + y$
-	Subtraction	Subtracts one value from another	$x - y$
*	Multiplication	Multiplies two values	$x * y$
/	Division	Divides one value by another	x / y
%	Modulus	Returns the division remainder	$x \% y$

++	Increment	Increases the value of a variable by 1	++x
--	Decrement	Decreases the value of a variable by 1	--x

Ex. 1) `int x = 5;`
`int y = 2;`
`printf("%d", x % y);`
`return 0;`

Output
→

1

2) `int x = 5;`
`printf("%d", ++x);`

Output
⇒

6

3) `int x = 5;`
`printf("%d", --x);`

Output
⇒

4

* modulus (%) operator
 ଲେଖିବା ଓ ଗଣନା କରିବା
 ଲେଖିବା
 ଉଦା: $2 \overline{) 5} \begin{array}{r} 2 \\ 4 \\ \hline 1 \end{array}$ ← ଫଳାଫଳ

Assignment operators

* variables ର values assign କରିବା ଲେଖିବା.

Operator	Example	Same as
=	<code>x = 5</code>	<code>x = 5</code>
+=	<code>x += 5</code>	<code>x = x + 5</code>
-=	<code>x -= 5</code>	<code>x = x - 5</code>
*=	<code>x *= 5</code>	<code>x = x * 5</code>
/=	<code>x /= 5</code>	<code>x = x / 5</code>
%=	<code>x %= 5</code>	<code>x = x % 5</code>
&=	<code>x &= 5</code>	<code>x = x & 5</code>
=	<code>x = 5</code>	<code>x = x 5</code>
^=	<code>x ^= 5</code>	<code>x = x ^ 5</code>
>>=	<code>x >>= 5</code>	<code>x = x >> 5</code>
<<=	<code>x <<= 5</code>	<code>x = x << 5</code>

Ex. 1) `int x = 3;`
`printf("%d", x += 5);` Output $\Rightarrow$ 8

2) `int x = 3;`
`printf("%d", x %= 2);` Output $\Rightarrow$ 1

Comparison operators

- * values and variables compare karhile (compare) kare.
- * Comparison operation karle kile (return value) kile 'Boolean' kare. (`true(1)` and `false(0)`)

Operator	Name	Example	Description
<code>==</code>	Equal to	<code>x == y</code>	Returns 1 if the values are equal
<code>!=</code>	Not Equal	<code>x != y</code>	Returns 1 if the values are not equal
<code>></code>	Greater Than	<code>x > y</code>	Returns 1 if the first value is greater than the second value.
<code><</code>	Less Than	<code>x < y</code>	Returns 1 if the first value is less than the second value.
<code>>=</code>	Greater Than or Equal to	<code>x >= y</code>	Returns 1 if the first value is greater than or equal to the second value.
<code><=</code>	Less Than or Equal to	<code>x <= y</code>	Returns 1 if the first value is less than or equal to the second value.

Ex. 1) `int x = 5;`
`int y = 3;`
`printf("%d", x > y);`

Output $\Rightarrow$ 1

* because 5 is greater than 3.
 So, returns 1 (true)


```
2) int x = 5;
   int y = 3;
   printf("%d", x <= y);
```

Output
⇒ 0

* because 5 is not less than or equal to 3.
So, returns 0 (false)

Logical Operators

* variables and values between which we are applying logical operators.

operator	Name	Example	Description
&&	AND	$x < 5 \ \&\& \ x < 10$	Returns 1 if both statements are true.
	OR	$x < 5 \ \ x < 4$	Returns 1 if one of the statements is true.
!	NOT	$!(x < 5 \ \&\& \ x < 10)$	Reverse the result, returns 0 if the result is 1.

Ex. 1) ~~int~~ int x = 5;
int y = 10;
printf("%d", (x < 10 && y < 20));

Output
⇒ 1

2) int x = 5;
int y = 10;
printf("%d", (x < 0 || y < 20));

Output
⇒ 1

* because x is not less than 0, but y is less than 20.
So, at least one statement is true.
Then, returns 1 (true)

* because x is less than 10 and y is less than 20.
So, both statements are true.
Then, returns 1 (true)

3) int x = 5;
int y = 10;
printf("%d", !(x < 0 || y < 20));

Output
⇒ 0

* result of (x < 0 || y < 20) is true(1).
So, returns false (0) because the operator is "NOT"

Note:

~~AND~~ AND operator $\rightarrow$ Statements હવે true હો જાય, output થશે true (1) છે.

OR operator $\rightarrow$ statements હવે એકે થશે true હો, output થશે true (1) છે.

NOT operator $\rightarrow$ statement થશે true હો; false (0) છે, false હો; true (1) છે, output થશે.

Booleans

- * C ની **bool** data type થી જાય. bool data type થી એકે થશે true અને false values હોય. થશે; true અને false છે.
- * bool data type થી int and char એકે **built-in-type** થી જાય છે. જો; કદાચ header થી જાય એકે import કરે જાય.

#include <stdbool.h>

- * boolean variables કદાચ જો declare કરે જાય.

E.x. 1.) bool isMarried = false;

2.) bool isTodayHot = true;

- * **boolean values return કરે એકે integer થી જાય.**
થશે;
① true થશે; 1 છે,
② false થશે; 0 છે, return કરે.

E.x. bool isMarried = false; Output
printf("%d", isMarried); $\Rightarrow$ 0

- * એકે જાય એકે comparison operators એકે statement થી જાય output થશે 1 છે, output થશે એકે return values થી 0 એકે 1 છે.

E.x. printf("%d", 10 > 9); // Returns 1 (true)

Complete example for bool type

```
#include <stdio.h>
#include <stdbool.h> // Import boolean header file

int main () {
    bool isPizzaTasty = true;
    printf("%d", isPizzaTasty == true);
    return 0;
}
```

Conditions and If Statements

- * කිසියම් කොන්දිෂන්ස් සත්‍ය වීම හෝ හොඳින් ප්‍රමාණය වීම; එබැවින්ම මෙය කොන්දිෂන්ස් ස්‍රෝතුවේ ප්‍රවණතාවයන් නිර්ණය වීමට, conditional statements යොදා ගනියි.
- * if යොදා ගන්නේ; කිසියම් code block එකක්, යම් condition එකක් true වන්නේ නම් නිවැරදිව execute වීම සඳහා ය.
- * else යොදා ගන්නේ; if වැනි කළේ condition එක false වීම යම් code block එකක් execute වීම සඳහා ය.
- * else if යොදා ගන්නේ; මුල් condition එක false වීම මෙය condition එකක් check කිරීම සඳහා ය.
- * switch යොදා ගන්නේ; විශාල වශයෙන් සුදුසු බව හා කිහිප code blocks කිහිපයක් execute කිරීම සඳහා ය.

if statements

- * යම් condition එකක් true වීම නිවැරදිව execute විය යුතු code block එකක් ප්‍රකාශයක් යොදා ගනියි.
- * මෙය පහත ප්‍රකාශනවලින් දැක්විය හැකිය.

```
if (condition) {  
    // block of code to be executed if the condition  
    is true  
}
```

E.x. `int x = 5;
int y = 10;
if (x < y) {
 printf("Yes, x is less than y");
}`

else statements

- * if statements නම් දිවුරුම් ලෙස යොදා ගනියි.
- * if statement එකක් condition එක false වීම if execute වේ.
- * මෙය පහත ප්‍රකාශනවලින් දැක්විය හැකිය.

```
if (condition) {  
    // block of code that executed if condition is true  
} else {  
    // block of code that executed if condition is false.  
}
```



```

Ex. int time = 20;
    if (time < 18) {
        printf("Good day!");
    } else {
        printf("Good evening!");
    }

```

Output
⇒ Good evening!

~~else...if statements~~
else...if statements

- * જ્યારે condition એક false થઈ; ત્યારે condition એકાંકી ૨, એક ૨ false થઈ ત્યારે એકાંકી ૩ conditions હિતો check કરીને ૩૨૨ code blocks execute થઈ શકાય છે. else...if statements આપે છે.
- * એક જ શરતો પરથી જણાઈ શકે છે.

```

if (condition1) {
    // block of code that executed when condition1 is true.
} else if (condition2) {
    // block of code that executed when condition2 is true
} else {
    // block of code that executed when all the above conditions are false.
}

```

```

Ex. int time = 22;
    if (time < 10) {
        printf("Good Morning");
    } else if (time < 20) {
        printf("Good day");
    } else {
        printf("Good evening");
    }

```

Output
⇒ Good evening

Note: if...else-if...else statement એકાંકી ૨, એક statement એક શરતો પરથી જોઈ શકાય છે. conditions check કરીને ૩૨૨; જો condition એકાંકી true થઈ તો એક જ શરતો થઈ શકે છે.
condition એકાંકી true થઈ તો ૩૨૨ code block એક execute થઈ શકે છે; જો જણાઈ તો code blocks execute થઈ જાય તો ૩૨૨ conditions check કરીને જોઈ શકે છે.

E.x. `int x = 10;`

```
if (x < 5) {  
    printf("x is less than 5");  
} else if (x < 15) {  
    printf("x is less than 15");  
} else if (x < 30) {  
    printf("x is less than 30");  
} else {  
    printf("x is 30 or greater than 30");  
}
```

Output
⇒ x is less than 15

⊛ જો $x < 15$ અને $x < 30$ ની conditions પૂરી થાય તો ~~જો~~ $x < 15$ ની condition પૂરી થાય તો `code block` એ execute થી; પ્રત્યેક `if - else... if - else` જોડોનું એકલું જ કાર્ય કરે છે.

Short hand if... else

- * if - else statement એક જ રીડેન્સન્ટ પ્રાથમિક સ્વરૂપ છે.
- * જેમાં $x < 15$ ની જગ્યાએ `condition` એકલું થાય છે.
- * જેમાં પ્રાથમિક અને સર્વિસ થાય છે.

`variable = (condition) ? expressionTrue : expressionFalse;`

E.x. `int time = 20;`

```
(time < 18) ? printf("Good day") : printf("Good evening");
```

↓ output

Good evening

Switch statements

- * if... else statements બદલે switch statement એકલું કાર્ય કરે છે.
- * જેમાં પ્રાથમિક અને સર્વિસ થાય છે.

```

switch (expression) {
    case x :
        // code block
        break;
    case y :
        // code block
        break;
    default :
        // code block
}

```

અન્યથા કહે

* switch (~~switch~~ expression)
expression એ, એક ચોક્કસ
મૂલ્ય હોય.

* case
expression એના value એ
કો case એના value
સાથ compare થાય.

① expression એ કો case
એ મળેલો હોય, તે case
એનો જુદો code block એ
execute થાય.

② break
execution એ અચોક્કસ
switch block એના જુદા
કોડ બ્લોક ચલાવે.

* default
પ્રોગ્રામરે 'ચોક્કસ' (optional)
દર્શાવે છે. કોઈ case
એના match ના પાડેલો
execute થાય.

break keyword એ જુદા
રૂપે છે. એક શરત પૂરી
હોય, condition એ
સંતોષિત થાય, કોઈ
cases execute થાય છે.

default keyword એ
optional છે. કોઈ case
એના expression એ
સાથ મળેલો હોય,
default નો એ code block
એ execute થાય છે.

Ex. 1) int day = 4 ;

```

switch (day) {
    case 1 :
        printf("Monday");
        break;
    case 4 :
        printf("Thursday");
        break;
    case 7 :
        printf("Sunday");
        break;
    default :
        printf("Any other day or Error");
        break;
}

```

2) int num = 3;

```

switch (num) {
    case 1 :
        printf("10");
        break;
    case 3 :
        printf("20");
    case 4 :
        printf("30");
}

```

/* Output
20 30 */

Loops

- * බැසිල් බොර්ලොස්කි කතෘ වන බොක්, එම ක්‍රියාවන් ග්‍රාහකයා සිදු කරීමට 100ps යොදයි.

- ★ loops are more readable code, save the time, reduce errors are advantages over if.

while Loop

- * while loop એક જગ્યાએ કોઈ કાર્ય પુનરાવરત કરવા વડી લો.

```
while (condition) {  
    // Code block to be executed  
}
```

```
Ex. int i = 0;
    while(i < 5) {
        printf("%d\n", i);
        i++;
    }
```

Output
⇒

- [illegible]

ଏକ ଗୋଟିଏ ଲୁପ୍, loop ଥିବା କିଛି କିଛି କ୍ଷେତ୍ର ଗୋଟିଏ!
(never ending loop)

காதுயிற் றீர்

- * print වන විට i හි value 0 වේ.
- * loop වන විට while condition වන check කරනු ලබයි; condition වන true වන විට, i හි value print කරයි.
- * $i++$ වන විට i හි value 1 වැනි වැඩි කරන (increment) කරයි; i හි value 1 වේ.
- * while වන විට i හි value 1 වැනි වැඩි කරන condition වන check කරයි.
- * while loop නිසි i හි value 5 ක් වැඩි වීම; condition වන false වන විට, loop වන execute වීම නැත.

do/while Loop

- ★ while loop એ છે એક પ્રકારનો.
- ★ do-while loop એના છે ક્યુએશન જેથી loop એક જુદા એ code block એક વારના execute કરે. છતાં પ્રશ્નરત condition એક check કરે.
- ★ જો એકવાર શરૂ; ~~જો~~ પ્રશ્નરત પ્રશ્નરતે છે એ condition એક false થાય છે, loop એકવાર એક code block એકવાર પ્રશ્નરત એક એક થઈ શરૂ કરે.
- ★ એક એક પ્રશ્નરત એક એક છે.

```
do {  
    // Code block to be executed  
}  
while (condition);
```

Ex. 1.) int i = 0;

```
do {  
    printf("%d\n", i);  
    i++;  
}  
while (i < 5);
```

Output
⇒ 0
1
2
3
4

2.) int i = 10;

```
do {  
    printf("%d\n", i);  
    i++;  
}  
while (i < 5);
```

Output
⇒ 10

★ એક છે i = 10 શરૂ, પ્રશ્નરતે છે condition એક false છે. એક; condition એક check કરે એક એક એક code block એક execute એક શરૂ, 10 output છે.

Note: while loop એકવાર છે પ્રશ્નરત condition એક check કરે. એક true જો એક code block એક execute છે.

do...while loop એકવાર છે પ્રશ્નરત code block એક (એક એક) execute થાય. છતાં પ્રશ્નરત condition એક check કરે.

For loop

- * loop એકવાર ક્રીયાશીલ થાય અને બરાબરી ચકાસવામાં આવે છે; for loop એકવાર ક્રીયાશીલ થાય.
- * એક જગ્યાએ ફક્ત એક જ વાર ચલિત થાય છે.

```
for (expression 1; expression 2; expression 3) {  
    // Code block to be executed  
}
```

- ① expression 1 $\Rightarrow$ એક code block એકવાર content થી execute થી શરૂ, પછી પછી execute થાય છે.
- ② expression 2 $\Rightarrow$ code block થી execute થી શરૂ condition થી define કરે છે.
- ③ expression 3 $\Rightarrow$ code block થી execute થી શરૂ, execute થાય અને એકવાર બરાબરી શરૂ થાય છે.

E.x. int i;

```
for (i=0; i<5; i++) {  
    printf ("%d\n", i);  
}
```

Output $\Rightarrow$

0
1
2
3
4

અર્થઘટન

- * પહેલાં i એ variable થી loop થી શરૂ થાય છે declare કરે છે. કે તે ફક્ત assign કરે છે.
- * પછી; loop થી શરૂ થી i 0 થી શરૂ થાય છે.
- * $i < 5$ condition થી check કરે છે. true થાય છે i output કરે છે.
- * પછી $i++$ થી i નું મૂલ્ય 1 થી વધે છે.
- * પછી $i=2$ થી શરૂ; કે તે પછી શરૂ થાય છે ક્રીયાશીલ થાય છે.
- * $i=5$ થી; condition થી false થાય છે, તેથી loop થી execute થી શરૂ થાય છે.

Nested Loops

- * જુઓ જુલોગિક ગતિ: loop એકાએક ચાલે અને loop એકાએક ચાલે એકાએક ચાલે. એક **nested loops** એક ચાલે.
- * એક જી જોઈએ જુઓ loop એક, "Outer loop" એક ૨; ૬૦ જુઓએ ૩ loop એક, "inner loop" એક ૨; ચાલે.

E.x. int i, j;

// Outer loop

for (i = 1; i <= 2; ++i) {

printf ("Outer : %d\n", i); // Executes 2 times

// Inner loop

for (j = 1; j <= 3; ++j) {

printf ("Inner : %d\n", j); // Executes 6 times (2x3)

}

}

↓ output

Outer : 1

Inner : 1

Inner : 2

Inner : 3

Outer : 2

Inner : 1

Inner : 2

Inner : 3

Break / Continue

- * જુઓ જી જી ૨, break keyword એક ચાલે અને એક જી switch statement એકાએક જુઓએ ચાલે.
- * એકાએક loop એકાએક જુઓએ ચાલે, break keyword એક ચાલે.
- ① જુઓ જુઓએ જુઓએ i જી જુઓએ 4 જી જી; loop એકાએક જુઓએ ચાલે અને જુઓએ જુઓએ જુઓએ જુઓએ for loop એકાએક.

E.x. int i;

```
for (i=0; i<10; i++) {  
    if (i==4) {  
        break;  
    }  
    printf("%d\n", i);  
}
```

Output
⇒

0
1
2
3

* **continue statement** એ loop થી બેક થઈ શકે છે, **break** કરતાં **continue** એ **breaks one iteration** થી.

* **break** નો ઉપયોગ એક loop થી બેક થઈ શકે છે, **continue** એ **breaks one iteration** થી.

E.x. int i;

```
for (i=0; i<10; i++) {  
    if (i==4) {  
        continue;  
    }  
    printf("%d\n", i);  
}
```

Output
⇒

0
1
2
3
5
6
7
8
9

★ એક 4 output નથી.

કે જેણે $i=4$ થી, loop થી **continue** કરે છે.

* **for loop** જોડવામાં આવે; **while loop** જોડવામાં આવે, **break** અને **continue** એકબીજાની સાથે.

Arrays

* **Arrays** એકબીજાની સાથે **variable** થી **values** સંગ્રહિત કરે છે.

* **Array** એક સંગ્રહ છે;

★ જે data type થી લેવામાં આવે છે.

★ **Array** નો **name** એક **square brackets** (**[]**) થી લેવામાં આવે છે.

★ **Array** નો **list** એક **comma separated** **list** થી લેવામાં આવે છે.

* Array એક જ પ્રકારના ઇલેમેન્ટ ધરાવે છે.

```
data type array name[] = {value 1, value 2, ...};
```

E.x. `int myNumbers[] = {25, 50, 75, 100};`

Note: Array એક જ પ્રકારના ઇલેમેન્ટના values ધરાવે data type એક જ હોય છે.

* Array એક જ પ્રકારના values, elements હોય છે. elements access કરવા index વાળું હોય છે.

* array indexes પ્રારંભ થાય છે 0 થી શરૂ;

first element $\Rightarrow$ index $\rightarrow 0$

second element $\Rightarrow$ index $\rightarrow 1$

E.x. `int myNumbers[] = {25, 50, 75, 100};`

`printf("%d", myNumbers[0]);` // Outputs 25

* array એક જ element એક જ હોય છે, indexing કરવા હોય છે.

E.x. `int myNumbers[] = {25, 50, 75, 100};`

`myNumbers[0] = 33;`

`printf("%d\n", myNumbers[0]);`

// Outputs 33 instead of 25

* array એક જ હોય છે loop એક જ હોય છે આ કારણે જ.

E.x. `int myNumber[] = {25, 50, 75, 100};`

`int i;`

`for (i = 0; i < 4; i++) {`

`printf("%d\n", myNumber[i]);`

`}`

↓ output

25

50

75

100

* Arrays create කිරීමේ විදිහ ප්‍රධාන ක්‍රමය වන්නේ; **ඉලුම array size එක ලබා දී, එහි elements add කිරීමයි.**

E.x. `int myNumbers[4];` // Declare an array of four integers

// Add elements

`myNumbers[0] = 25;`

`myNumbers[1] = 50;`

`myNumbers[2] = 75;`

`myNumbers[3] = 100;`

* Array එකක **size එක ලබා ගැනීම sizeof operator** යොදා.

E.x. `int myNumbers[] = {10, 25, 50, 75, 100};`

`printf("%lu", sizeof(myNumbers));` // Outputs 20

⊕ මෙහි `sizeof` array size (array length) එක 5 වුව ද, output කරන්නේ 20 වේ.

මේ හේතුව; ඉන් ඉස්ලාමයක දැක්වූ පරිදි, `sizeof` operator මගින් එන්ටරයක කරන්නේ memory size එක byte ගණනක් ලෙසයි.

`int` type එකක දූලේ memory size එක 4 ක් වනිනි, $4 \times 5 = 20$ ලෙස 20 පෙන් වේ.

* නිවැරදිව array එකක දූන් elements ගණන එකවර output කිරීමට ඉහත ඉංකාරයක් යොදා ගනිමු.

E.x. `int myNumbers[] = {10, 25, 50, 75, 100};`

`int length = sizeof(myNumbers) / sizeof(myNumbers[0]);`

`printf("%d", length);` // Outputs 5

Strings

* **strings** යොදාගන්නේ **text data** store කිරීම සඳහාය.

* strings යොදාගන්නේ **double quotation marks** දූලයි.

* දැනට බොහෝ programming languages වල ඉංකාරයට C වල දී **string data type** එකක් ගොඩනගනු ලැබේ.

මෙම හේතුව නිසා; දූරට **string** එකක් සෙවීමට ඉඩකඩ තව;

char type එකෙහි array එකක් තිබෙන නිසා, එයට

string value එක සෙවීමට සිදු වේ.

E.x. char greetings[] = "Hello World";

④ એ છે string એ double quotes માં લેવામાં આવે છે.

★ આ પ્રકારના પ્રકારે printf() function એ કોઈપણ format specifier લેઈ %s લેવામાં આવે છે.

④ એથી જ આ પ્રકારે strings કોઈ કોઈપણ compiler માં ચલાવવામાં આવે છે.

★ strings એ C માં એક એકલકો array એ છે; arrays માં indexing લેવામાં આવે છે characters output આપે છે.

E.x. char greetings[] = "Hello World";

printf("%c", greetings[0]); // Outputs H

Note: format specifier લેઈ %C લેવામાં આવે છે એક character એકલકો output આપે છે.

★ arrays માં elements modify કરવા માટે strings માં એક છે, એ છે.

E.x. char greetings[] = "Hello World";

greetings[0] = 'J';

printf("%s", greetings);

// Outputs Jello World instead of Hello World

★ arrays માં string એકલકો કારણ છે, loops લેવામાં આવે છે.

E.x. char carName[] = "Volvo";

int i;

for (i = 0; i < 5; ++i) {

printf("%c\n", carName[i]);

}

Output
⇒

V
o
l
v
o

★ strings એકલકો આપે છે એકલકો પ્રકારના પ્રકારે.

④ એ છે characters set એકલકો લેઈ string એકલકો store કરવામાં આવે છે.

E.x. char greetings[] = {'H', 'e', 'l', 'l', 'o', ' ', 'W', 'o', 'r', 'l', 'd', '\0'};

★ આ characters એકલકો પ્રકારના 10 escape character એકલકો લેવામાં આવે છે. આ એકલકો string એકલકો store કરવામાં આવે છે એક પ્રકારના છે. એકલકો પ્રકારના, string એકલકો પ્રકારના એકલકો છે. (null terminating char)

String functions

- * C 𑀓𑀓𑀓 string functions 𑀓𑀓𑀓 𑀓𑀓𑀓𑀓𑀓𑀓 𑀓𑀓. strings 𑀓𑀓 𑀓𑀓𑀓𑀓𑀓 𑀓𑀓𑀓𑀓𑀓 𑀓𑀓𑀓𑀓𑀓 𑀓𑀓 𑀓𑀓𑀓𑀓𑀓 𑀓𑀓𑀓𑀓𑀓 𑀓𑀓𑀓.
- * 𑀓𑀓 𑀓𑀓𑀓𑀓 𑀓𑀓𑀓 <string.h> header file 𑀓𑀓 programme 𑀓𑀓𑀓 import 𑀓𑀓𑀓𑀓 𑀓𑀓𑀓.
- * string 𑀓𑀓𑀓 length 𑀓𑀓 𑀓𑀓𑀓𑀓 strlen() function 𑀓𑀓 𑀓𑀓𑀓𑀓.

E.x. #include <stdio.h>
#include <string.h>

```
int main () {  
    char alpha[] = "Hello";  
    printf("%d", strlen(alpha));  
    return 0;  
}
```

Output
⇒ 5

Note: 𑀓𑀓 𑀓𑀓𑀓𑀓𑀓 𑀓𑀓 𑀓𑀓 sizeof 𑀓𑀓𑀓𑀓𑀓 string 𑀓𑀓𑀓 size 𑀓𑀓 𑀓𑀓𑀓𑀓 𑀓𑀓𑀓𑀓.

𑀓𑀓 sizeof 𑀓𑀓 strlen 𑀓𑀓𑀓 𑀓𑀓𑀓𑀓 𑀓𑀓𑀓𑀓𑀓 𑀓𑀓𑀓𑀓 𑀓𑀓𑀓𑀓.

sizeof 𑀓𑀓 𑀓𑀓 10 character 𑀓𑀓 𑀓 𑀓𑀓𑀓𑀓 𑀓𑀓𑀓 𑀓𑀓𑀓;
strlen 𑀓𑀓 𑀓𑀓 𑀓𑀓 𑀓𑀓𑀓𑀓 𑀓𑀓𑀓𑀓.

E.x. char alpha[] = "Hello";
printf("%d", strlen(alpha)); // Outputs 5
printf("%d", sizeof(alpha)); // Outputs 6

- * strings 𑀓𑀓𑀓𑀓 𑀓𑀓 𑀓𑀓 𑀓𑀓𑀓𑀓 combine 𑀓𑀓𑀓 "concatenation" 𑀓𑀓𑀓 𑀓𑀓𑀓𑀓.
- * 𑀓𑀓 𑀓𑀓𑀓 strcat() function 𑀓𑀓 𑀓𑀓𑀓𑀓 𑀓𑀓𑀓.

E.x. char str1[20] = "Hello";
char str2[] = "World";

// Concatenate str2 to str1 (result is stored in str1)
strcat(str1, str2);

// Output str1
printf("%s", str1);

Note: ເພື່ອ ຄຳຂໍ້ມູນຂອງ ຂໍ້ str1[20] ເປັນ square brackets ທີ່
ເຮັດໄດ້ ກ່ຽວກັບ str1 ທີ່ ຖືກຕ້ອງ length ຂອງມັນ.

⊛ ເພື່ອ length ຂອງ str1 ກັບ str2 ເຊັ່ນ ຂອງ ຂໍ້ມູນ ທີ່
ຖືກຕ້ອງ str1 ມີ ສະຖານະ ຂອງ ຂໍ້ມູນ; ຂໍ້ ຄຳຂໍ້ມູນ
character lengths ທີ່ ຂໍ້ ຂໍ້ມູນ ຂອງ ຂໍ້ມູນ length
ຂອງ ຂໍ້ມູນ ຂອງ ຂໍ້ມູນ.

★ ຂໍ້ string ຂອງ value ຂອງ ຂໍ້ມູນ string ຂອງ ຂໍ້ມູນ copy ຂໍ້ມູນ
strcpy() function ຂອງ ຂໍ້ມູນ.

E.x. char str1[20] = "Hello";
char str2[20];

```
// Copy str1 to str2  
strcpy(str2, str1);  
printf("%s", str2);
```

★ strings compare ຂໍ້ມູນ strcmp() function ຂອງ ຂໍ້ມູນ.

- ⊛ strings ຂອງ ຂໍ້ມູນ ຂໍ້ມູນ; 0 return ຂໍ້ມູນ.
- ⊛ ຂໍ້ມູນ ຂໍ້ມູນ; ຂໍ້ມູນ ຂໍ້ມູນ return ຂໍ້ມູນ.

E.x. char str1[] = "Hello";
char str2[] = "Hello";
char str3[] = "world";

```
printf("%d\n", strcmp(str1, str2)); // Returns 0  
printf("%d\n", strcmp(str1, str3)); // Return other number,  
but not 0
```

User Input

★ ຂໍ້ຂໍ້ມູນ ຂອງ ຂໍ້ມູນ ຂອງ ຂໍ້ມູນ ຂອງ programme ຂອງ ຂໍ້ມູນ
ຂໍ້ມູນ ຂໍ້ມູນ user inputs ຂໍ້ມູນ ຂໍ້ມູນ ຂໍ້ມູນ ຂໍ້ມູນ.

★ user input ຂອງ ຂໍ້ມູນ ຂໍ້ມູນ scanf() function ຂອງ ຂໍ້ມູນ.

★ ຂໍ້ຂໍ້ມູນ ຂໍ້ມູນ ຂໍ້ມູນ.

```
printf("Type a number : \n"); //Ask the user to type a  
                                number
```

```
scanf("%d", &myNum); //Get and save the number the user types.
```

```
printf("Your number is : %d", myNum); //Output the number
```

Note: (★) scanf() function එක arguments ලෙසින් ලබා ගනියි.

(1) format specifier of the variable (%d)

(ii) reference operator which stores the memory address of the variable (&myNum)

* `scanf()` function accepts data types which is given in the format specifier, user inputs are given in the format specifier.

E.x. `int myNum;`
`char myChar;`

```
printf("Type a number and a character and press enter : \n");
```

```
scanf("%d %c", &myNum, &myChar);
```

```
printf("Your number is : %d \n", myNum);
```

```
printf("Your character is: %c \n", myChar);
```

* String inputs ඉහත ආකාරී වන අර්ථ දැක්වීමක් ඇත.

E.x. // Create a string
char firstName[30];

```
// Ask the user to input some text
```

```
printf("Enter your first name: \n");
```

```
//Get and save the text
```

```
scanf ("%s", firstName);
```

```
// Output the text
```

```
printf("Hello %s", firstName);
```

Note: scanf() function strings string කරනු ලබන නිසා, string/array size එක (හෝ 1000000000) දැනටමත් තීරණය කළ යුතුය; එය user විසින් input කරන string එක store කරගැනීමට තරම් ප්‍රමාණවත් තරමක් විය යුතුය.

④ $\otimes$ ରେ $\otimes$ reference operator ($\otimes$) କෙଉଁଠି ଉପସ୍ଥାନ କରାଯାଇଛି.